

LECTURE 06

Logistic Regression

NATURAL LANGUAGE PROCESSING
Dr. Tamal Ghosh

TOPIC

Probabilistic Discriminative Learning

SUBTITLE

From Linear Scores to Probabilities

STATUS

Core Foundations

Where Are We Now?

LECTURE 03

Text → Vectors

BoW / TF-IDF



LECTURE 04

Probabilistic Classification

Naïve Bayes



LECTURE 05

Discriminative Classification

Perceptron



TODAY

Logistic Regression

Probabilistic Discriminative

Learning Objectives



Discriminative Classification

UNDERSTAND LOGISTIC REGRESSION'S ROLE IN THE ML ECOSYSTEM.



Probabilistic Confidence

CONVERT RAW LINEAR SCORES INTO MEANINGFUL CONFIDENCE LEVELS.



NLP Application

APPLY THE MODEL TO REAL-WORLD TEXT CLASSIFICATION TASKS.



The Sigmoid Function

MASTER THE MATH BEHIND SQUASHING SCORES TO PROBABILITIES.



Optimization Logic

EXPLAIN THE MECHANICS OF LOG-LOSS AND GRADIENT DESCENT.

The Perceptron Revisited

LINEAR SCORE CALCULATION

$$z = w \cdot x + b$$

IF $Z \geq 0$

Positive Class

IF $Z < 0$

Negative Class

The Problem

The Perceptron provides a **hard decision**. It lacks a measure of confidence.

"How certain is the model about this specific prediction?"

Hard Decision vs Probability

PERCEPTRON

Prediction: POSITIVE

A **binary, rigid** output. The model is either right or wrong, no middle ground.

LOGISTIC REGRESSION

$P(\text{Positive} \mid x) = 0.92$

A **probabilistic, nuanced** output. We get the prediction AND the confidence level.

92% CONFIDENCE

The Linear Score (z)

GENERAL FOUNDATION

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

NUMERICAL EXAMPLE

Features (x): [2, 3]

Weights (w): [1, 0.5]

Bias (b): -1

CALCULATION

$$z = 1(2) + 0.5(3) - 1$$

$$z = 2 + 1.5 - 1$$

$$z = 2.5$$

Why Do We Need Sigmoid?

THE PROBABILITY CONSTRAINT

A valid probability must lie strictly between 0 and 1.

LINEAR SCORE (Z)

$(-\infty, +\infty)$

SQUASH



SIGMOID

PROBABILITY (P)

$(0, 1)$

Linear scores can be any real number (e.g., -100, 0, 10.5). We need a function that maps this infinite range to the finite probability space.

The Sigmoid Function Formula

$$\sigma(z) = 1 / (1 + e^{-z})$$

$$(-\infty, +\infty) \rightarrow (0, 1)$$

$$z \rightarrow -\infty$$

$$\sigma(z) \rightarrow 0$$

$$z = 0$$

$$\sigma(z) = 0.5$$

$$z \rightarrow +\infty$$

$$\sigma(z) \rightarrow 1$$

Sigmoid Intuition

Large Negative Score

e.g., $z = -10$



Probability near 0

Strong Negative Confidence

Score = 0

Neutral Point



Probability = 0.5

Maximum Uncertainty

Large Positive Score

e.g., $z = +10$



Probability near 1

Strong Positive Confidence

Think of the sigmoid as a "Probability Converter" for linear scores.

Sigmoid Examples Table

Score (z)	Probability $\sigma(z)$	Confidence Level
-5	≈ 0.007	High Negative
-2	≈ 0.119	Moderate Negative
-1	≈ 0.269	Low Negative
0	0.500	Neutral / Uncertain
1	≈ 0.731	Low Positive
2	≈ 0.881	Moderate Positive
5	≈ 0.993	High Positive

Key Observation: Positive scores ($z > 0$) always map to probabilities > 0.5 , while negative scores ($z < 0$) map to probabilities < 0.5 .

The Logistic Regression Model

STEP 1: LINEAR SUM

$$z = w \cdot x + b$$

STEP 2: ACTIVATION

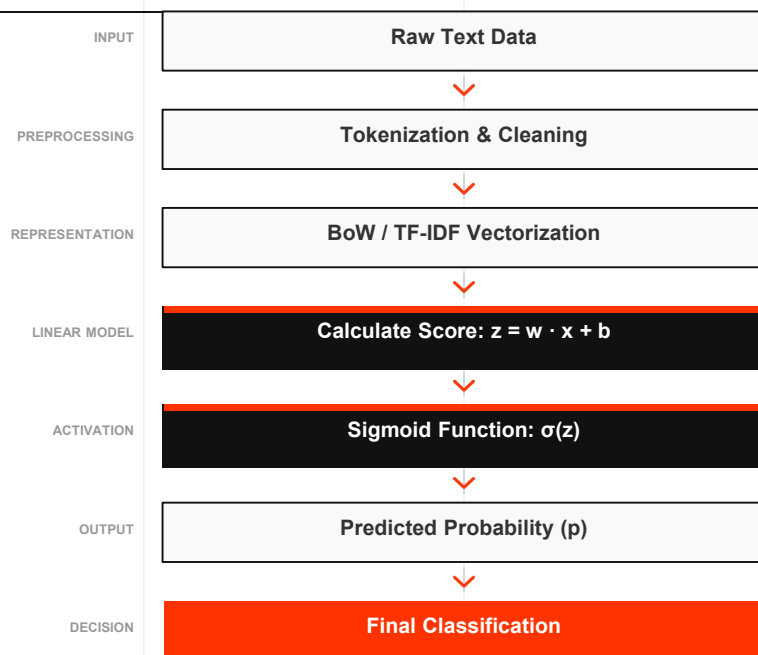
$$p = \sigma(z)$$

UNIFIED PROBABILISTIC MODEL

$$P(y=1|x) = 1 / (1 + e^{-(w \cdot x + b)})$$

This complete equation directly computes the **probability** that a given input belongs to the positive class, based on learned weights and features.

Logistic Regression Pipeline



NLP in Action: A Sentiment Example

INPUT SENTENCE

"This movie is fantastic."

VECTORIZATION

$x = [0, 1, 0, 1, 1, \dots]$

LINEAR SCORE

$z = w \cdot x + b = 2.75$

SIGMOID

$\sigma(2.75) = 0.94$

FINAL PREDICTION

94% POSITIVE

Sentiment Classification & Thresholds

CASE A: HIGH PROBABILITY

$P(\text{Positive}) = 0.94$

$0.94 \geq 0.5$

POSITIVE

CASE B: LOW PROBABILITY

$P(\text{Positive}) = 0.27$

$0.27 < 0.5$

NEGATIVE

Decision Rule: Convert continuous probability into binary class labels.

The Decision Threshold

DEFAULT

0.5

Balanced approach for standard classification tasks.

CONSERVATIVE

0.8

Only predicts Positive when very confident.
Avoids False Positives.

AGGRESSIVE

0.2

Eager to predict Positive. Avoids False
Negatives.

REAL-WORLD APPLICATION In **medical screening** or **spam detection**, the threshold is often adjusted to prioritize either sensitivity or specificity based on the cost of errors.

Prediction Is Not Enough

SCENARIO 1

Actual Label (y) 1

Prediction (p) 0.90

GOOD PREDICTION

SCENARIO 2

Actual Label (y) 1

Prediction (p) 0.10

TERRIBLE PREDICTION

How wrong is the model?

WE NEED A WAY TO QUANTIFY ERROR TO GUIDE LEARNING.

The Loss Function

A loss function quantifies the error between the model's prediction and the actual truth.

LOG LOSS / BINARY CROSS-ENTROPY

$$L = -[y \log(p) + (1-y)\log(1-p)]$$

y

Actual Label (0 or 1)

p

Predicted Probability

Why Log Loss?

SCENARIO A: GOOD PREDICTION

Actual (y): 1

Predicted (p): 0.95

MINIMAL LOSS

SCENARIO B: CONFIDENT MISTAKE

Actual (y): 1

Predicted (p): 0.01

EXTREME PENALTY

Log Loss heavily penalizes predictions that are **Wrong and Confident**.

Log Loss Intuition

Actual Label: **Positive** ($y = 1$)

Predicted Probability (p)	Loss Value ($-\log p$)	Error Level
0.99	0.01	Minimal
0.90	0.10	Small
0.50	0.69	Moderate
0.10	2.30	High
0.01	4.60	Extremely High

THE PENALTY LOGIC: As the prediction moves further from the truth, the loss increases exponentially. The model is **HEAVILY PENALIZED** for being wrong and confident.

The Training Objective

STEP 01

Initialize Weights

STEP 02

Make Prediction

STEP 03

Calculate Loss

STEP 04

Adjust Weights

THE ULTIMATE GOAL

Minimize Total Loss

Gradient Descent Optimization

Loss



GLOBAL MINIMUM

THE ALGORITHM

An iterative optimization algorithm that repeatedly adjusts parameters to reduce the loss.

THE INTUITION

Move parameters in the direction that decreases the loss function most steeply.

"Imagine a ball rolling down a hill to reach the lowest point—the point—the minimum possible error."

The Learning Rate (η)

SMALL LEARNING RATE

Tiny Steps

Learning is very safe and stable, but extremely slow. It may take a long time to reach the minimum loss.

LARGE LEARNING RATE

Huge Steps

Learning is fast, but risky. The model may "overshoot" the minimum and fail to converge or even diverge.

Finding the **Optimal Balance** is critical for efficient model training.

Gradient Descent Update Rule

PARAMETER UPDATE FORMULA

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L$$

θ

Model Parameters (Weights & Bias)

η

Learning Rate (Step Size)

∇L

Gradient (Slope of the Loss)

We subtract the gradient to move in the opposite direction of the slope, effectively rolling "downhill" toward the minimum loss.

Intuition Behind the Gradient

"Which direction should I move this weight to reduce the error?"

THE GRADIENT'S CORE QUESTION

High Loss



Calculate Gradient



Update Weights



Lower Loss

Think of the gradient as a **compass**. By repeating this cycle thousands of times, the model "learns" the optimal configuration for classification.

The Complete Training Pipeline

STAGE 01

Training Docs → Preprocessing → Vectors



STAGE 02

Feature Matrix X → Initialize w, b



STAGE 03

Scores $z = w \cdot x + b$ → Sigmoid Probabilities



STAGE 04

Log Loss → Calculate Gradients



STAGE 05

Update Parameters via Gradient Descent



STAGE 06

Repeat until model converges

Interpreting Feature Weights

Word	Weight (w)	Influence
fantastic	+3.2	Strong Positive
excellent	+2.8	Strong Positive
boring	-2.1	Strong Negative
terrible	-3.0	Strong Negative

POSITIVE WEIGHTS

Increase the linear score (z), pushing the sigmoid output toward 1 (Positive Class).

NEGATIVE WEIGHTS

Decrease the linear score (z), pushing the sigmoid output toward 0 (Negative Class).

Logistic Regression vs. Perceptron

Feature	Perceptron	Logistic Regression
MODEL TYPE	Linear Classifier	Linear Classifier
OUTPUT	Hard Decision (0/1)	Probability (0 to 1)
ACTIVATION	Step Function	Sigmoid Function
LOSS	Perceptron Update	Log Loss (Cross-Entropy)
CONFIDENCE	None / Limited	High (Calibrated)

Key Difference: While both are linear classifiers, Logistic Regression provides a **nuanced, probabilistic confidence** that is critical for real-world NLP applications.

Logistic Regression vs. Naïve Bayes

Feature	Naïve Bayes	Logistic Regression
PARADIGM	Generative	Discriminative
ASSUMPTION	Feature Independence	No Independence Assumption
TRAINING	Counting / Estimation	Iterative Optimization
PERFORMANCE	Fast, Robust on Small Data	High Accuracy, Calibrated
BOUNDARY	Linear	Linear

Generative vs. Discriminative: Naïve Bayes models the joint probability $P(x,y)$, while Logistic Regression directly models the conditional $P(y|x)$, making it more flexible for complex feature interactions.

Python Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
# 1. Vectorize Text
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(texts)
# 2. Train Model
model = LogisticRegression()
model.fit(X, labels)
# 3. Predict Probability
probs = model.predict_proba(test_vector)
print(f"Confidence: {probs}")
>> Confidence: [[0.06, 0.94]]
```

STEP 01

Vectorization

STEP 02

Model Fitting

STEP 03

Probabilistic Prediction

Summary & Classroom Challenge

KEY TAKEAWAYS

Logistic Regression is a **probabilistic discriminative classifier**.

The **Sigmoid function** is the core engine for probability conversion.

Log Loss and **Gradient Descent** drive the iterative learning process.

Quick Challenge

If linear score $z = 2$:

$\sigma(2) \approx 0.88$ (High Positive)

If linear score $z = -2$:

$\sigma(-2) \approx 0.12$ (High Negative)